

14b — Suchalgorithmen

Linear vs. Binär — warum sich Sortieren lohnt

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Suchalgorithmen?
2. Lineare Suche – die naive Variante
3. Binäre Suche – Halbieren statt Durchlaufen
4. Vergleich: bei $n=1\,000\,000$ ist binär ca. 50 000-mal schneller
5. Wann braucht man welche?

Lernziel: Sie können lineare und binäre Suche selbst implementieren, ihre Voraussetzungen nennen und ihre Komplexität begründen.

Wie wir heute arbeiten: Sehen Sie Suche im Terminal – mit Cursor- und Intervall-Visualisierung.

Wozu Suchalgorithmen?

Veggie Soles – 10 000 Produkte im Katalog. Wo ist “Hemp-High”?

katalog = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot", ...] # 10

Anwendungsfall	Beispiel
Produktkatalog	Finde Artikel an Position i
Telefonbuch	Finde Nummer zu Name
Datenbank	Finde Datensatz mit ID
Freitext-Suche	Wort in Dokument finden

Heute: zwei Algorithmen, **gleiches Ergebnis**, sehr **unterschiedliche Geschwindigkeit**.

Lineare Suche - die Idee

- Gehe die Liste **von vorne nach hinten** durch
- Vergleiche jedes Element mit dem Ziel
- Erstes Treffer-Match → Position zurückgeben
- Kein Treffer → -1 zurückgeben

```
def lineare_suche(liste, ziel):  
    for index, element in enumerate(liste):  
        if element == ziel:  
            return index  
    return -1
```

Wert	Komplexität
Zeit (best/avg/worst)	$O(1)$ / $O(n)$ / $O(n)$
Speicher	$O(1)$
Voraussetzung an Liste	keine

Lineare Suche - *sehen*

Index:	0	1	2	3	4
Liste:	[Eco,	Hemp,	Bam,	Sock,	Schn]
		↑pos			

Index:	0	1	2	3	4
Liste:	[Eco,	Hemp,	Bam,	Sock,	Schn]
		↑pos			

<- Treffer!

Demo zeigt nach jedem Vergleich die Liste mit Cursor ↑pos unter dem aktuellen Index.

Visualisierungsstufe 4a: Zeiger-Cursor – Vorbereitung auf Binärsuche, wo wir gleich drei Zeiger gleichzeitig sehen.

► **Live-Demo** - 02_lineare_suche_cursor.py

Binäre Suche - Voraussetzung

Die **Liste muss sortiert sein.**

unsortiert = [5, 1, 3, 2, 4] # *NICHT geeignet*
sortiert = [1, 2, 3, 4, 5] # *geeignet*

Hier zahlt sich der Aufwand aus 14a aus: einmal sortieren → viele schnelle Suchen.

- Anwendung: in Datenbanken werden Indizes (sortierte Strukturen) genau dafür gepflegt.
- Wenn die Liste sich oft ändert: einmal sortieren ist teuer – Trade-off.

Binäre Suche - die Idee

Telefonbuch in der Mitte aufschlagen, dann nur in der relevanten Hälfte weitersuchen.

Suche 7 in [1, 2, 3, 4, 5, 6, 7, 8]

↑mid (=4)

7 > 4 -> rechts

[5, 6, 7, 8]

↑mid (=6)

7 > 6 -> rechts

[7, 8]

↑mid (=7)

Treffer!

Bei jedem Schritt **halbiert** sich der Suchraum. Aus $n=1024$ wird in maximal **10 Schritten** ein Treffer.

Binäre Suche - der Code

```
def binaere_suche(liste, ziel):  
    links, rechts = 0, len(liste) - 1  
    while links <= rechts:  
        mitte = (links + rechts) // 2  
        if liste[mitte] == ziel:  
            return mitte  
        elif liste[mitte] < ziel:  
            links = mitte + 1  
        else:  
            rechts = mitte - 1  
    return -1
```

Wert	Komplexität
Zeit (best/avg/worst)	$O(1)$ / $O(\log n)$ / $O(\log n)$

Binäre Suche - das Intervall *sehen*

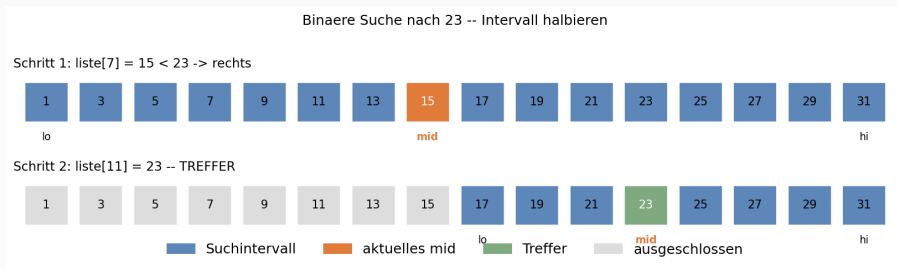


Abbildung 1: Binäre Suche – Intervall halbieren

Demo druckt vor jedem Schritt: - die Liste - drei Marker $\uparrow lo$, $\uparrow mid$, $\uparrow hi$ unter den Indizes - den Vergleich Ziel vs. `liste[mid]`

Visualisierungsstufe 4b: drei Zeiger gleichzeitig. Man sieht das Intervall in jedem Schritt um die Hälfte schrumpfen.

Vergleich: linear vs. binär

n	Lineare Suche (worst)	Binäre Suche (worst)
100	100 Schritte	7 Schritte
10 000	10 000 Schritte	14 Schritte
1 000 000	1 000 000 Schritte	20 Schritte
1 000 000 000	1 Milliarde Schritte	30 Schritte

Bei $n=1$ Mrd. ist binär ca. **30 Millionen-mal** schneller – aber nur, wenn die Liste sortiert ist.

► **Live-Demo** - 04_linear_vs_binaer.py

Wann nehme ich welche?

Situation	Empfehlung
Liste klein ($n < 100$)	Lineare Suche - der Unterschied ist nicht spürbar
Liste oft durchsucht, selten geändert	Sortieren + binäre Suche
Liste nur einmal benutzt	Lineare Suche - Sortieren ist teurer als die eine Suche
Daten in Dictionary / Set	ziel in d - Hash-basiert, im Schnitt $O(1)$
Daten in Datenbank	DB-Engine nutzt sortierte Indizes intern

In Python ist wert in liste lineare Suche, wert in dict ist Hash-Suche - ein riesiger Unterschied!

Cheat Card

Algorithmus	Voraussetzung	Zeit (worst)	Idee
Lineare Suche	keine	$O(n)$	Element für Element prüfen
Binäre Suche	sortierte Liste	$O(\log n)$	Halbiere bei jedem Schritt
in auf Liste	keine	$O(n)$	intern lineare Suche
in auf Dict / Set	keine	$O(1)$ avg	Hash-basiert

Faustregel: Wer suchen will, soll sortieren - oder gleich ein dict/set verwenden.

Wir haben jetzt ein solides Algorithmen-Fundament:

- 13a: Algorithmen modellieren (Pseudocode, UML)
- 13b: Komplexität (Big-O)
- 14a: Sortieren (Bubble, Insertion, Quick, TimSort)
- 14b: Suchen (linear, binär)

In NB 15 lernen Sie, wie man eigenen Code in **Module** verpackt – damit Algorithmen wiederverwendbar werden.

Ab NB 16: echte Datenanalyse mit Pandas und Matplotlib – dort werden Sortieren und Suchen täglich gebraucht.

Heute geübt

- ✓ Lineare Suche mit Cursor-Visualisierung verfolgt
- ✓ Binäre Suche mit Intervall-Visualisierung verstanden
- ✓ Komplexitäten $O(n)$ vs. $O(\log n)$ konkret erlebt
- ✓ Voraussetzung "sortierte Liste" als zentralen Punkt erkannt
- ✓ Praxis-Tipp: dict/set für $O(1)$ -Lookup

Zur Vertiefung im Notebook 14b:

- Implementieren Sie `lineare_suche` und `binaere_suche` selbst
- Vergleichen Sie ihre Performance auf 1 Mio. Elementen
- Recherche: warum nutzen Datenbanken **B-Trees** für Indizes? (Hinweis: ähnlich wie binäre Suche, aber für Festplatten optimiert)